

□ Data Types & Distributions

Normal Distribution • All Chart Types • Skewness & Kurtosis • Python Code
Beginner-Friendly Guide for Data Science & Machine Learning

□ What This Chapter Covers

- ✓ Types of Data — Categorical vs Numerical (and sub-types)
- ✓ Normal / Gaussian Distribution — what it is, why it matters, its limitations
- ✓ Other major distributions — Uniform, Bernoulli, Binomial, Poisson, Exponential, T
- ✓ The 68-95-99.7 Rule — fully explained
- ✓ How to detect if data is normally distributed
- ✓ How to normalize data — methods and Python code
- ✓ All chart types — Histogram, Box Plot, Bar Chart, Scatter, and more
- ✓ Skewness & Kurtosis — explained in full detail
- ✓ Complete Python code for every concept

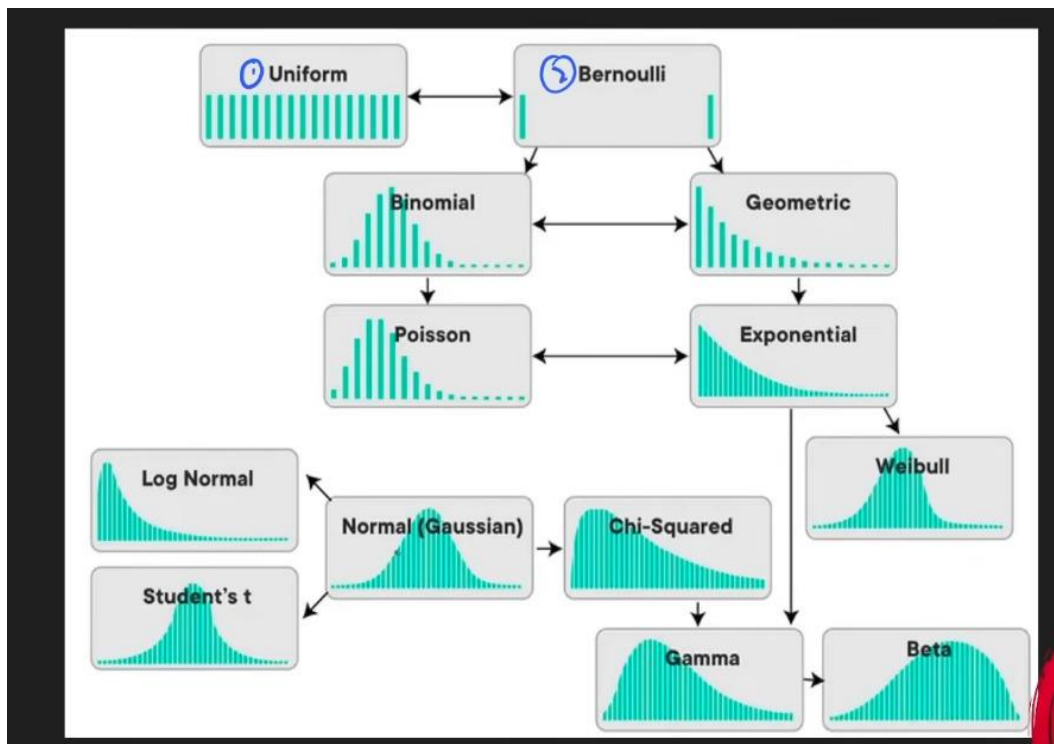


Figure 1: How all probability distributions relate to each other

1 □ Types of Data

□ Why Data Types Matter

Before you can choose a chart, distribution, or analysis method, you MUST know what TYPE of data you are working with.

The wrong type assumption → wrong chart → wrong model → wrong conclusions.

□ Categorical Data — Data with Labels or Groups

Categorical data represents qualities or labels — things you can PUT INTO GROUPS. You cannot do math on these (you cannot add "Red" + "Blue").

Sub-type	What it is	Examples	Python dtype
Nominal	Labels with NO order — one is not "more" than another	Blood type (A/B/O), Colours, Country, Gender	object / str / pd.Categorical
Ordinal	Labels WITH order — but gaps between ranks are not equal	Star ratings (1★–5★), Education level, Survey scale (Bad/OK/Good)	pd.Categorical(ordered=True)

□ Nominal vs Ordinal — Easy Trick

Nominal = No order. Red is not "more" than Blue.

Ordinal = Has order. 5 stars IS more than 1 star. But is 5★ FIVE TIMES better than 1★? Not necessarily.

The key: ordinal has RANK but NOT equal spacing between ranks.

□ Numerical Data — Data with Actual Numbers

Numerical data represents measurements or counts. You CAN do math on these.

Sub-type	What it is	Examples	Key property
Discrete	Countable whole numbers — you CANNOT have half a value	Number of students, Number of goals, Coin flips	Finite or countably infinite gaps between values
Continuous	Any value in a range — infinite possibilities between two points	Height (1.731 m), Temperature, Weight, Time	Can take any decimal value — no gaps
Interval	Numbers WITH equal spacing but NO true zero point	Temperature in °C (0°C is NOT "no temperature"), Years	Can add/subtract, CANNOT multiply/divide meaningfully

Sub-type	What it is	Examples	Key property
Ratio	Numbers WITH equal spacing AND a true zero point	Height, Weight, Speed, Income (0 = truly nothing)	Full math allowed — ratios make sense (2× as tall)

Interval vs Ratio — The Zero Point Test

Ask: Does zero mean TRULY NOTHING?

Temperature 0°C = NOT "no temperature". It's just freezing point. → INTERVAL

Height 0 cm = truly no height. Zero means nothing. → RATIO

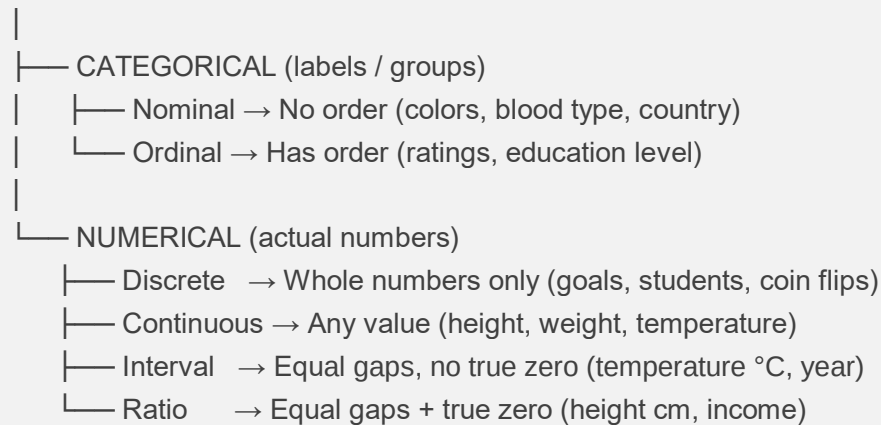
Income Rs. 0 = truly no income. → RATIO

Only RATIO lets you say "2× as much". You cannot say 20°C is twice as hot as 10°C!

Complete Data Type Map

DATA TYPE TREE

DATA



Python — Identifying Data Types

Python Code

```

import pandas as pd
import numpy as np

df = pd.DataFrame({
    'name'      : ['Ali', 'Sana', 'Bilal'],    # Nominal categorical
    'grade'     : ['A', 'B', 'A'],            # Ordinal categorical
    'score'     : [85, 72, 90],                # Discrete numerical
    'height_cm' : [170.5, 162.3, 175.8],       # Continuous numerical
    'temp_c'    : [36.6, 37.1, 36.9],         # Interval
    'income_pkr': [50000, 80000, 120000],     # Ratio
})
  
```

```
print(df.dtypes)      # shows int64, float64, object
print(df.describe())  # stats for numerical columns only

# Convert to ordered categorical:
df['grade'] = pd.Categorical(df['grade'], categories=['C','B','A'], ordered=True)
print(df['grade'] > 'B')  # True/False - ordering works now
```

2 □ Normal / Gaussian Distribution

□ What IS Normal Distribution?

Normal Distribution (also called Gaussian Distribution) describes data that clusters around a central value (the mean) and gets rarer and rarer as you move further away from that center — forming a **perfect bell-shaped curve**.

Think of it this way: if you measured the height of 10,000 people, most would be around the average. Very few would be extremely short or extremely tall. That pattern — concentrated in the middle, tapering at the extremes — IS the Normal Distribution.

□ Key Properties of Normal Distribution

- ✓ Symmetric — the left and right halves are perfect mirror images
- ✓ Mean = Median = Mode — all three averages are the same point
- ✓ Bell-shaped — unimodal (single peak) at the center
- ✓ Defined by two parameters only: μ (mean = center) and σ (std dev = width)
- ✓ Total area under the curve = 1 (100% of all probability)
- ✓ Extends to infinity in both directions, but tails become negligibly small

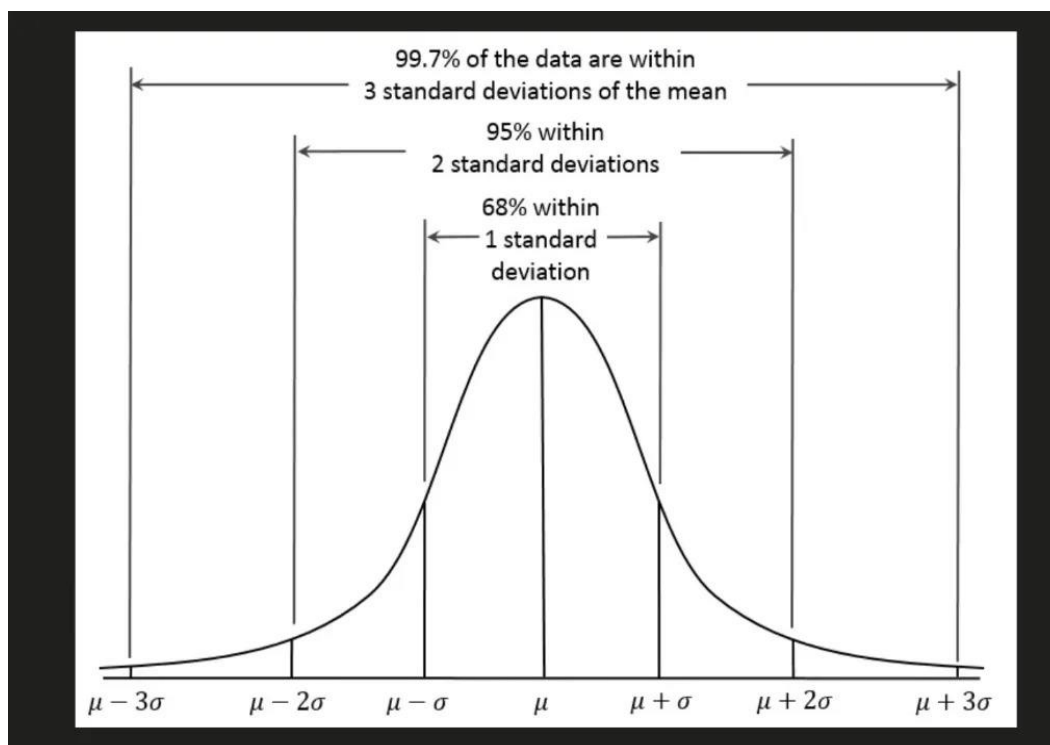


Figure 2: Normal Distribution — the Bell Curve showing $\mu \pm 1\sigma$, 2σ , 3σ

□ The Formula (Understand, Don't Memorise)

Normal Distribution Formula

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \times \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right)$$

Where:

μ (mu) = mean → controls the CENTER (where the peak is)

σ (sigma) = std dev → controls the WIDTH (how spread out it is)

x = any data point you are evaluating

Larger σ → bell curve is WIDER and FLATTER

Smaller σ → bell curve is NARROWER and TALLER

□ The 68-95-99.7 Rule (Empirical Rule)

This is one of the most used rules in all of statistics and ML. Memorise it.



Figure 3: The Empirical Rule — 68%, 95%, 99.7% of data within 1, 2, 3 standard deviations

Range	% of Data	In your data	Real-life meaning
$\mu \pm 1\sigma$	68%	Between (mean - std) and (mean + std)	About 2 in 3 students fall within 1 std of the average
$\mu \pm 2\sigma$	95%	Between (mean - 2×std) and (mean + 2×std)	19 out of 20 values fall here — used for 95% confidence intervals
$\mu \pm 3\sigma$	99.7%	Between (mean - 3×std) and (mean + 3×std)	Almost everything. Values outside are extreme outliers (0.3%)

□ Real-Life Example — Exam Scores

Exam scores: Mean (μ) = 70, Std Dev (σ) = 10

68% of students scored between 60 and 80 ($\mu \pm 1\sigma$)

95% of students scored between 50 and 90 ($\mu \pm 2\sigma$)

99.7% of students scored between 40 and 100 ($\mu \pm 3\sigma$)

If someone scores 95, they are more than 2σ above mean → top 2.5% of class.

This is how Z-scores work — measuring HOW MANY standard deviations from the mean.

□ Why Is Normal Distribution Important in ML?

Where It Appears	Why It Matters
Central Limit Theorem	Sample means follow Normal distribution regardless of original data shape. This is WHY statistical tests work.
Linear Regression	Assumes errors (residuals) are normally distributed. If not, predictions are unreliable.
Neural Networks	Weights are initialized from Normal distribution. Random init avoids symmetry breaking problems.
Feature Scaling	StandardScaler assumes Normal dist. It shifts mean to 0 and std to 1. Only works well if data is Normal.
Confidence Intervals	The 95% CI formula uses $\pm 1.96\sigma$ — directly from the Normal distribution's 95% rule.
Hypothesis Testing	Z-tests and t-tests assume Normal distribution of test statistics.

❑ Limitations of Normal Distribution

Normal Distribution is powerful, but it is NOT always appropriate. These are the situations where it breaks down:

❑ When Normal Distribution Does NOT Apply

- ✗ Skewed data — income, house prices, population — heavily right-skewed, NOT Normal
- ✗ Count data — number of goals, emails per hour — use Poisson instead
- ✗ Binary outcomes — spam/not-spam, click/no-click — use Bernoulli/Binomial
- ✗ Heavy-tailed data — financial returns, earthquake magnitudes — fat tails, not Normal
- ✗ Multimodal data — if you see two humps, it's bimodal, not one Normal curve
- ✗ Very small samples — with $n < 30$, use t-distribution (heavier tails, more uncertainty)
- ✗ Bounded data — percentages must be in $[0,1]$; Normal extends to $-\infty$ and $+\infty$

❑ What To Do When Data Is Not Normal

- Apply Log Transform, Box-Cox, or Yeo-Johnson to make it more Normal
- Use Robust Scaling (median/IQR) instead of StandardScaler
- Use non-parametric tests (Mann-Whitney, Kruskal-Wallis) instead of t-tests
- Choose the correct distribution (Poisson for counts, Binomial for binary events)

❑ How to Know if Your Data is Normally Distributed?

In real ML work you rarely have perfectly normal data. Here are the tests:

Method	What it does	Python
Histogram	Visual check — should look like a bell	<code>plt.hist(data, bins='auto')</code>
Q-Q Plot	Points should follow diagonal line if Normal	<code>stats.probplot(data, plot=plt)</code>
Shapiro-Wilk Test	$p > 0.05$ → data is Normal (best for small samples)	<code>stats.shapiro(data)</code>
K-S Test	Kolmogorov-Smirnov test for larger samples	<code>stats.kstest(data, 'norm')</code>
Skewness/Kurtosis	Skew ≈ 0 and Kurtosis ≈ 0 means Normal	<code>stats.skew(data),</code> <code>stats.kurtosis(data)</code>

❑ Python Code

```
from scipy import stats
import numpy as np
import matplotlib.pyplot as plt

data = np.random.normal(70, 10, 200)
```



```

# — Method 1: Histogram —————
plt.hist(data, bins=20) # look for bell shape
plt.title('Is this Normal? – check the shape')
plt.show()

# — Method 2: Q-Q Plot —————
fig, ax = plt.subplots()
stats.probplot(data, dist='norm', plot=ax)
ax.set_title('Q-Q Plot – points on line = Normal')
plt.show()

# — Method 3: Shapiro-Wilk Test —————
stat, p_value = stats.shapiro(data)
print(f'Shapiro-Wilk: stat={stat:.4f}, p={p_value:.4f}')
if p_value > 0.05:
    print('✓ Data is likely NORMAL')
else:
    print('✗ Data is NOT normal')

# — Method 4: Skewness & Kurtosis —————
skew = stats.skew(data)
kurt = stats.kurtosis(data) # excess kurtosis (Normal = 0)
print(f'Skewness : {skew:.4f} (should be near 0 for Normal)')
print(f'Kurtosis : {kurt:.4f} (should be near 0 for Normal)')

```

□ Python — Working with Normal Distribution

□ Python Code

```

import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

# — Generate normally distributed data —————
mu, sigma = 70, 10
data = np.random.normal(mu, sigma, 1000)

# — Basic stats —————
print(f'Mean : {np.mean(data):.2f}') # ≈ 70
print(f'Std : {np.std(data, ddof=1):.2f}') # ≈ 10

# — 68-95-99.7 Rule check —————
within_1sd = np.sum((data > mu-sigma) & (data < mu+sigma)) / len(data)
within_2sd = np.sum((data > mu-2*sigma) & (data < mu+2*sigma)) / len(data)
within_3sd = np.sum((data > mu-3*sigma) & (data < mu+3*sigma)) / len(data)
print(f'Within 1σ: {within_1sd*100:.1f}%') # ≈ 68%
print(f'Within 2σ: {within_2sd*100:.1f}%') # ≈ 95%

```

```
print(f'Within  $3\sigma$ : {within_3sd*100:.1f}%') #  $\approx 99.7\%$ 

# — Plot the bell curve —————
plt.figure(figsize=(10, 5))
plt.hist(data, bins=40, density=True, alpha=0.6, color='steelblue')
x = np.linspace(mu-4*sigma, mu+4*sigma, 300)
plt.plot(x, stats.norm.pdf(x, mu, sigma), 'r-', lw=2, label='Normal PDF')
plt.axvline(mu, color='black', linestyle='--', label=f'Mean={mu}')
plt.axvline(mu+sigma, color='orange', linestyle='--', label=' $\pm 1\sigma$ ')
plt.axvline(mu-sigma, color='orange', linestyle='--')
plt.legend()
plt.title('Normal Distribution - Bell Curve')
plt.show()
```

3 Other Types of Distributions

Discrete vs Continuous Distributions

DISCRETE: Data can only take specific separate values (whole numbers).

Examples: Binomial, Bernoulli, Poisson, Geometric

Think: Number of goals, coin flip outcomes, number of emails per hour

CONTINUOUS: Data can take any value in a range (infinite possibilities).

Examples: Normal, Exponential, Uniform (continuous), T-distribution

Think: Height, weight, time, temperature

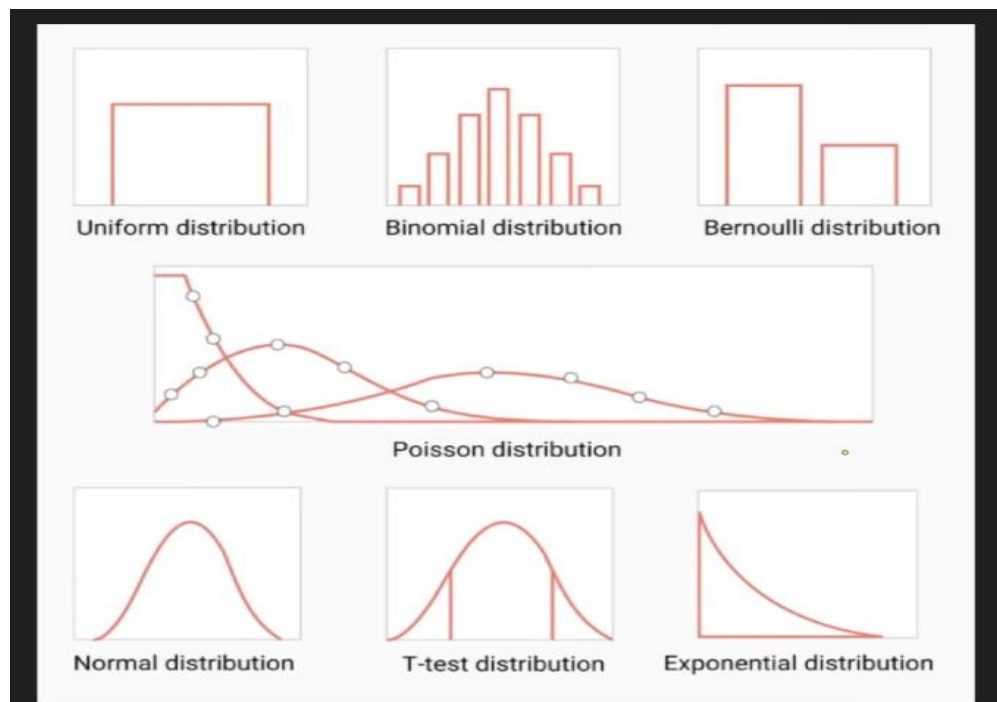


Figure 4: Types of Probability Distributions at a glance

3.1 — Uniform Distribution

Every outcome has the SAME probability. Flat, equal chance for everything.

Real-Life Analogy

Rolling a fair die: 1, 2, 3, 4, 5, 6 — each has exactly $1/6$ chance.

Picking a random number between 1 and 10 — every number equally likely.

Random number generator in computers — perfectly uniform.

Discrete Uniform	Continuous Uniform
Finite outcomes — each has equal probability Example: fair die (1–6)	Any value in [a, b] equally likely Example: random number in [0, 1]

Python Code

```
import numpy as np, matplotlib.pyplot as plt

# Discrete Uniform — rolling a die 1000 times
die_rolls = np.random.randint(1, 7, 1000)
plt.hist(die_rolls, bins=6, rwidth=0.8, color='steelblue')
plt.title('Discrete Uniform — Die Rolls'); plt.show()

# Continuous Uniform — between 0 and 10
data = np.random.uniform(0, 10, 1000)
plt.hist(data, bins=20, color='teal')
plt.title('Continuous Uniform Distribution'); plt.show()
```

3.2 — Bernoulli Distribution

The SIMPLEST distribution — only TWO possible outcomes: SUCCESS (1) or FAILURE (0). One single trial. One event. One probability p .

Real-Life Analogy

Flipping one coin: Heads (1) or Tails (0).

One patient: Recovered (1) or Not recovered (0).

One email: Spam (1) or Not spam (0).

One click: Clicked (1) or Not clicked (0).

Bernoulli(p) — p is the probability of success (1).

If $p=0.7 \rightarrow$ 70% chance of success, 30% chance of failure.

Python Code

```
from scipy import stats
p = 0.7 # probability of success (heads)

# Single Bernoulli trial
outcome = stats.bernoulli.rvs(p) # 0 or 1
print(f'Outcome: {outcome}') # e.g., 1 (heads)

# 1000 coin flips
flips = stats.bernoulli.rvs(p, size=1000)
print(f'Heads: {flips.sum()}, Tails: {1000-flips.sum()}')
```

```
# In ML: this is exactly what Logistic Regression outputs!  
# Output probability → threshold at 0.5 → Bernoulli outcome (0 or 1)
```

3.3 — Binomial Distribution

Binomial = MULTIPLE Bernoulli trials. How many successes in n trials?

□ Real-Life Analogy

Flip a coin 10 times. How many heads do you get?
That count (0 to 10) follows a Binomial distribution.

Binomial(n, p):

n = number of trials

p = probability of success in each trial

X = number of successes (this is what we find)

Bernoulli = special case of Binomial where n=1.

□ Python Code

```
from scipy import stats  
import numpy as np, matplotlib.pyplot as plt  
  
n, p = 10, 0.5 # 10 coin flips, fair coin  
  
# Probability of exactly 6 heads  
prob_6 = stats.binom.pmf(6, n, p)  
print(f'P(exactly 6 heads) = {prob_6:.4f}') # 0.2051  
  
# Probability of 6 or fewer heads  
prob_leq6 = stats.binom.cdf(6, n, p)  
print(f'P(6 or fewer heads) = {prob_leq6:.4f}') # 0.8281  
  
# Simulate 1000 experiments of 10 flips each  
results = stats.binom.rvs(n, p, size=1000)  
plt.hist(results, bins=range(12), rwidth=0.8, color='steelblue')  
plt.title('Binomial(n=10, p=0.5) - 1000 Simulations'); plt.show()
```

3.4 — Poisson Distribution

How many times does an event happen in a FIXED time period? Used when events happen randomly but at a known average rate (λ , lambda).

□ Real-Life Analogy

You receive on average 5 emails per hour ($\lambda = 5$).

What is the probability you get exactly 3 emails in the next hour?

Other examples:

Number of cars passing a toll booth per minute

Number of goals in a football match

Number of server crashes per day

Key conditions: events are independent, constant average rate.

□ Python Code

```
from scipy import stats
import numpy as np, matplotlib.pyplot as plt

lam = 5 # average 5 emails per hour

# Probability of exactly 3 emails
prob_3 = stats.poisson.pmf(3, lam)
print(f'P(3 emails) = {prob_3:.4f}') # 0.1404

# Plot Poisson distribution for different lambdas
x = np.arange(0, 20)
for lam in [1, 4, 10]:
    plt.plot(x, stats.poisson.pmf(x, lam), marker='o', label=f' $\lambda={lam}$ ')
plt.title('Poisson Distribution - different  $\lambda$  values')
plt.legend(); plt.show()
```

3.5 — T-Distribution (Student's t)

Similar to Normal distribution but with HEAVIER tails — used when sample size is SMALL ($n < 30$).

□ Real-Life Analogy

You want to estimate the average height of all Pakistanis.

But you only measured 15 people (small sample).

With a small sample, you are LESS certain about the true mean.

The t-distribution accounts for this extra uncertainty — its tails are wider.

As sample size grows → t-distribution becomes Normal.

Used in: t-tests (comparing means), confidence intervals with small samples.

□ Python Code

```
from scipy import stats
import numpy as np, matplotlib.pyplot as plt

x = np.linspace(-4, 4, 300)

# Compare Normal vs t-distribution
plt.plot(x, stats.norm.pdf(x), label='Normal (infinite df)', lw=2)
plt.plot(x, stats.t.pdf(x, df=3), label='t (df=3)', lw=2, linestyle='--')
plt.plot(x, stats.t.pdf(x, df=10), label='t (df=10)', lw=2, linestyle='-.')
plt.title('t-distribution vs Normal'); plt.legend(); plt.show()

# One-sample t-test: is sample mean different from  $\mu=70$ ?
sample = np.random.normal(72, 10, 20) # small sample (n=20)
t_stat, p_value = stats.ttest_1samp(sample, popmean=70)
print(f't-statistic: {t_stat:.4f}')
print(f'p-value      : {p_value:.4f}')
if p_value < 0.05:
    print('Significantly different from 70')
```

3.6 — Exponential Distribution

How long until the NEXT event happens? Time between events. Continuous counterpart of Poisson. Memoryless — the past doesn't affect future.

□ Real-Life Analogy

You know customers arrive at 5 per hour on average (Poisson).

How long do you WAIT until the next customer? → Exponential!

More examples:

How long until the next earthquake? (given average rate)

How long until the next server request?

Battery lifetime — how long until failure?

Parameter: λ (rate). Mean waiting time = $1/\lambda$.

If $\lambda=5$ (5 per hour), mean wait = $1/5$ hour = 12 minutes.

□ Python Code

```
from scipy import stats
import numpy as np, matplotlib.pyplot as plt
```

```
lam = 5 # 5 customers per hour → scale = 1/lam = 12 minutes

# Probability of waiting more than 20 minutes (1/3 hour)
prob_gt20 = 1 - stats.expon.cdf(1/3, scale=1/lam)
print(f'P(wait > 20 min) = {prob_gt20:.4f}') # 0.1889

# Simulate wait times
wait_times = stats.expon.rvs(scale=1/lam, size=1000)
plt.hist(wait_times, bins=40, color='salmon')
plt.xlabel('Wait Time (hours)')
plt.title('Exponential Distribution - Time Between Customers')
plt.show()
```


4 How to Normalize Data — Methods & Python Code

What is Normalization / Standardization?

ML models get confused when features have very different scales.

Example: Age (20–80) vs Income (10,000–5,000,000) — income dominates by scale alone.

Normalization: rescales data to a standard range (usually 0–1).

Standardization: rescales data to have mean=0 and std=1.

These are the main tools to "fix" your data before feeding it to ML models.

Method	Formula	Output Range	Best For
Min-Max Normalization	$(x - \min) / (\max - \min)$	[0, 1]	When you know data has fixed bounds; Neural Networks
Z-score Standardization	$(x - \text{mean}) / \text{std}$	Unbounded, mean=0, std=1	When data is approximately Normal; most ML algorithms
Robust Scaling	$(x - \text{median}) / \text{IQR}$	Unbounded, outlier-resistant	When data has many outliers
Log Transform	$\log(x)$ or $\log(x+1)$	Unbounded	Right-skewed data; income, prices
Sqrt Transform	$\sqrt{x}$	Unbounded	Moderately skewed data, count data
Box-Cox Transform	Finds best λ automatically	Near-Normal	When you don't know which transform to use
Yeo-Johnson	Like Box-Cox but allows zeros/negatives	Near-Normal	Box-Cox but data has 0 or negative values

Python Code

```
import numpy as np, pandas as pd, matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler, StandardScaler, RobustScaler, PowerTransformer
from scipy import stats

# Sample skewed data (like income)
np.random.seed(42)
raw = np.random.exponential(scale=10, size=500).reshape(-1, 1)

# — Min-Max Normalization —
data_mm = MinMaxScaler().fit_transform(raw).flatten()

# — Z-score Standardization —
```

```
data_std = StandardScaler().fit_transform(raw).flatten()

# — Robust Scaling —————
data_rob = RobustScaler().fit_transform(raw).flatten()

# — Log Transform —————
data_log = np.log1p(raw.flatten()) # log(x+1) handles zeros

# — Box-Cox (sklearn PowerTransformer) —————
data_bc = PowerTransformer(method='box-cox').fit_transform(raw).flatten()

# — Yeo-Johnson —————
data_yj = PowerTransformer(method='yeo-johnson').fit_transform(raw).flatten()

# — Check if result is Normal after transform —————
for name, d in [('Log', data_log), ('Box-Cox', data_bc)]:
    stat, p = stats.shapiro(d[:50])
    print(f'{name}: p={p:.4f} — {"Normal" if p>0.05 else "Not Normal"}')
```

5▣ Skewness & Kurtosis — Shape Descriptors

When you look at a distribution, the mean and standard deviation tell you WHERE it is centered and HOW SPREAD OUT it is. But they do NOT tell you the SHAPE. Skewness and Kurtosis fill that gap — they describe the shape of the distribution beyond what mean and std capture.

▣ Skewness — Is It Leaning?

Skewness measures the **asymmetry** of a distribution. A perfectly symmetric distribution has skewness = 0. If the data leans to one side, it has non-zero skewness.

▣ The Simple Intuition

Imagine balancing a histogram on your finger.

If it balances perfectly in the middle → Skewness = 0 (Normal)

If the right tail is longer (data dragged to the right) → Positive Skew

If the left tail is longer (data dragged to the left) → Negative Skew

The TAIL direction tells you the sign of skewness.

The Three Types of Skewness

Type	Skewness Value	Shape Description	Real-World Examples	Mean vs Median
Symmetric (Normal)	Skewness ≈ 0	Perfect bell curve. Left and right tails are equal.	Heights, IQ scores, exam scores in a balanced class	Mean $\approx$ Median $\approx$ Mode
Positive / Right Skew	Skewness > 0	Long tail on the RIGHT. Most data is on the LEFT (low values), but a few very high values pull the tail right.	Income distribution (most people earn low-medium, a few billionaires pull right), House prices, Stock returns	Mean $>$ Median $>$ Mode
Negative / Left Skew	Skewness < 0	Long tail on the LEFT. Most data is on the RIGHT (high values), but a few very low values pull the tail left.	Exam scores when most students do very well but a few fail badly, Age at retirement	Mean $<$ Median $<$ Mode

□ Memory Trick — Which Way Does the Tail Go?

Positive Skew = tail points to POSITIVE (right) side of the number line.

Negative Skew = tail points to NEGATIVE (left) side of the number line.

Right-skewed data: mean gets pulled toward the outliers on the right.

So: Mean > Median (the mean is dragged right by outliers)

Left-skewed data: mean gets pulled toward the outliers on the left.

So: Mean < Median (the mean is dragged left by outliers)

This is why for income data (right skewed) we use MEDIAN salary, not MEAN.

The mean is inflated by a few very high earners and is misleading!

□ Skewness Formula

Skewness Formula

$$\text{Skewness} = \frac{1}{n} \sum_{i=1}^n \frac{(x_i - \bar{x})^3}{s^3}$$

Where:

x_i = each data point

$\bar{x}$ = sample mean

s = standard deviation

n = number of data points

The cube (³) is critical: it preserves the SIGN (positive or negative), unlike variance which squares everything and loses direction.

□ Interpreting Skewness Values

Skewness Value	Interpretation	Action Needed
-0.5 to +0.5	Approximately symmetric — treat as Normal	None. Z-score standardization is fine.
-1.0 to -0.5 or +0.5 to +1.0	Moderately skewed — some asymmetry	Consider log or sqrt transform.
< -1.0 or > +1.0	Highly skewed — strong asymmetry	Apply log transform, Box-Cox, or use robust methods.

□ Real-World Example — Pakistan House Prices

Why House Price Data is Right-Skewed

Most houses in Pakistan are priced between Rs. 50 lakh and Rs. 2 crore.
But a small number of luxury houses sell for Rs. 20–100 crore.

Histogram: Most data bunched on the left (affordable homes).
Long tail stretches to the right (luxury homes).

Mean house price: Rs. 1.5 crore (inflated by luxury homes)
Median house price: Rs. 80 lakh (much more representative)

- Skewness is POSITIVE (right-skewed)
- Use MEDIAN to describe "typical" house price, not mean
- Apply log transform before using this data in ML models

□ Python Code

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

# — Simulate Right-Skewed Data (like income) —————
right_skewed = np.random.exponential(scale=10, size=1000)

# — Simulate Left-Skewed Data (like easy exam scores) ———
left_skewed = 100 - np.random.exponential(scale=10, size=1000)

# — Symmetric Normal Data —————
normal_data = np.random.normal(50, 10, 1000)

# — Calculate Skewness —————
for name, data in [("Normal", normal_data),
                  ("Right Skew", right_skewed),
                  ("Left Skew", left_skewed)]:
    skewness = stats.skew(data)
    print(f'{name:12s}: skewness = {skewness:.3f}')

# — Plot All Three —————
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
datasets = [("Normal (skew≈0)", normal_data, "steelblue"),
            ("Right Skew (+)", right_skewed, "salmon"),
            ("Left Skew (-)", left_skewed, "mediumseagreen")]

for ax, (name, data, color) in zip(axes, datasets):
    ax.hist(data, bins=30, color=color, edgecolor="white")
    ax.axvline(np.mean(data), color="red", linestyle="--", label="Mean")
    ax.axvline(np.median(data), color="black", linestyle="-", label="Median")
```

```
ax.set_title(f"{name}\nSkew={stats.skew(data):.2f}")
ax.legend(fontsize=8)

plt.suptitle("Skewness Comparison", fontsize=14, fontweight="bold")
plt.tight_layout()
plt.show()

# — Fixing Skewness with Log Transform —————
log_data = np.log1p(right_skewed)
print(f'Before log: skew = {stats.skew(right_skewed):.3f}')
print(f'After log: skew = {stats.skew(log_data):.3f}') # much closer to 0
```

❑ Kurtosis — Are the Tails Heavy or Light?

Kurtosis measures the **"tailedness" or peakedness** of a distribution. It tells you: how likely are extreme values (outliers)? Are the tails heavier or lighter than a Normal distribution?

❑ The Simple Intuition

Kurtosis answers: "Compared to a Normal curve, do the tails have MORE or LESS data?"

High Kurtosis → heavy tails → MORE outliers than expected from Normal data

Low Kurtosis → light tails → FEWER outliers, data is more uniformly spread

A Normal distribution has kurtosis = 3 (or excess kurtosis = 0 in Python's scipy).

Python reports EXCESS kurtosis = actual kurtosis – 3.

So in Python: Normal excess kurtosis = 0.

The Three Types of Kurtosis

Type	Kurtosis (excess)	Shape Description	Real-World Examples
Mesokurtic	≈ 0 (excess)	Normal bell curve. Standard tail weight. Reference point.	Heights, IQ scores — anything that follows Normal distribution
Leptokurtic	> 0 (excess)	Tall, sharp peak + very heavy tails. More data near the center AND more extreme outliers. Think: narrow but with fat tails.	Financial returns (stock markets crash occasionally), Seismic data, Cybersecurity attack frequencies
Platykurtic	< 0 (excess)	Flat, wide peak + very thin tails. Data is more	Uniform distribution, exam scores when most students get

Type	Kurtosis (excess)	Shape Description	Real-World Examples
		uniformly spread, fewer outliers than Normal.	similar scores (no extremes)

❑ Why Kurtosis Matters in Finance (Critical!)

Stock market returns are LEPTOKURTIC — they have heavier tails than Normal.

This means: extreme events (crashes like 2008, COVID crash 2020) happen MUCH MORE OFTEN than a Normal distribution would predict.

If a financial model assumes Normal distribution (kurtosis = 0):

It drastically underestimates the probability of a market crash.

This is called "fat tail risk" or "tail risk".

Many financial firms lost billions because their models assumed Normal tails.

Always check kurtosis before using Normal distribution in finance!

❑ Kurtosis Formula

Kurtosis Formula

$$\text{Kurtosis} = \frac{1}{n} \sum_{i=1}^n \frac{(x_i - \bar{x})^4}{s^4} - 3$$

The "-3" gives us EXCESS kurtosis (Normal = 0). This is what scipy returns.

Why power of 4?

The 4th power makes all values POSITIVE (like 2 in variance).

It amplifies extreme values much more than 2 does.

So kurtosis is very sensitive to outliers.

Kurtosis $\gg 0 \rightarrow$ many extreme values (heavy tails)

Kurtosis $\ll 0 \rightarrow$ few extreme values (light tails)

❑ Interpreting Kurtosis Values

Excess Kurtosis	Type	Interpretation	Example Distribution
≈ 0	Mesokurtic	Normal tail behavior. Outliers occur at expected frequency.	Normal / Gaussian

Excess Kurtosis	Type	Interpretation	Example Distribution
> 0 (e.g., 3–7)	Leptokurtic	Heavy tails. More outliers than expected. Peaked center.	Student's t-distribution (small df), Laplace distribution
> 7	Extremely Leptokurtic	Very heavy tails. Extreme events are common. High risk.	Stock market returns during volatility, earthquake magnitudes
< 0 (e.g., -1 to -2)	Platykurtic	Light tails. Fewer outliers than Normal. Data is more spread evenly.	Uniform distribution, Beta(2,2)

Skewness vs Kurtosis — Key Differences

Aspect	Skewness	Kurtosis
What it measures	Asymmetry / lean direction of the distribution	Tail heaviness / how outlier-prone the distribution is
Normal value	0 (symmetric)	0 excess (Python) / 3 actual
Positive value means	Right tail is longer (data skewed right)	Heavier tails than Normal (more outliers)
Negative value means	Left tail is longer (data skewed left)	Lighter tails than Normal (fewer outliers)
Affected by	The direction of outliers	The frequency/size of outliers (all sides)
Use case	Decide if log transform is needed, choose mean vs median	Assess tail risk, check if Normal assumption holds

Python Code

```

import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

# — Generate three different kurtosis distributions —
normal = np.random.normal(0, 1, 5000) # Mesokurtic
t_dist = np.random.standard_t(df=3, size=5000) # Leptokurtic (heavy tails)
uniform = np.random.uniform(-3, 3, 5000) # Platykurtic (light tails)

# — Print skewness and kurtosis for each —
for name, data in [("Normal", normal),
                  ("T-dist (df=3)", t_dist),
                  ("Uniform", uniform)]:
    sk = stats.skew(data)
    kurt = stats.kurtosis(data) # excess kurtosis (Normal = 0)
    print(f"{name:18s}: Skew={sk:6.3f} | Kurtosis(excess)={kurt:7.3f}")

```



```

# — Combined: Skewness + Kurtosis check —————
def describe_shape(data, name="Data"):
    skewness = stats.skew(data)
    kurtosis = stats.kurtosis(data)
    print(f"\n=== {name} ===")
    print(f"  Skewness : {skewness:.3f}")
    if abs(skewness) < 0.5:
        print("    → Approximately symmetric")
    elif skewness > 0:
        print("    → Positively skewed (right tail) – consider log transform")
    else:
        print("    → Negatively skewed (left tail)")
    print(f"  Kurtosis : {kurtosis:.3f} (excess)")
    if abs(kurtosis) < 0.5:
        print("    → Normal tail behavior")
    elif kurtosis > 0:
        print("    → Heavy tails – MORE outliers than Normal")
    else:
        print("    → Light tails – FEWER outliers than Normal")

income = np.random.exponential(scale=50000, size=2000)
describe_shape(income, "Income Data")

# — Plot: Normal vs Leptokurtic vs Platykurtic —————
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
configs = [
    ("Mesokurtic\n(Normal, Kurt≈0)", normal, "steelblue"),
    ("Leptokurtic\n(t-dist, Kurt>0)", t_dist, "tomato"),
    ("Platykurtic\n(Uniform, Kurt<0)", uniform, "mediumseagreen"),
]
for ax, (title, data, color) in zip(axes, configs):
    ax.hist(data, bins=60, density=True, color=color, alpha=0.7, edgecolor="white")
    x = np.linspace(-5, 5, 300)
    ax.plot(x, stats.norm.pdf(x), "k--", lw=1.5, label="Normal ref")
    ax.set_title(f"{title}\nKurt={stats.kurtosis(data):.2f}")
    ax.set_xlim(-5, 5)
    ax.legend(fontsize=8)

plt.suptitle("Kurtosis Comparison", fontsize=14, fontweight="bold")
plt.tight_layout()
plt.show()

```

□ Using Skewness & Kurtosis Together

In real data analysis, you check both Skewness and Kurtosis together to fully understand the shape of your data before choosing ML models or transformations.

Skewness	Kurtosis (excess)	Shape Description	Recommendation
≈ 0	≈ 0	Normal — symmetric, standard tails	Safe to use StandardScaler, Z-score
+ve (right skew)	High (> 1)	Right-skewed with heavy tails — income-like	Apply log transform, use Robust Scaler
-ve (left skew)	High (> 1)	Left-skewed with heavy tails	Reflect then log, or use Yeo-Johnson
≈ 0	Very high (> 3)	Symmetric but heavy-tailed — financial data	Use t-distribution, check for outliers
≈ 0	Negative (< -1)	Flat distribution — uniform-like	Min-Max scaling, consider Uniform model

★ The Full Shape Check Workflow

Step 1: Plot a histogram → get a visual sense of the shape

Step 2: Calculate Skewness → is it leaning? Which direction?

$|\text{skew}| < 0.5 \rightarrow \text{OK}$; $0.5\text{--}1.0 \rightarrow \text{moderate}$; $> 1.0 \rightarrow \text{high}$ — needs transform

Step 3: Calculate Kurtosis → are the tails heavy or light?

$\text{Kurt} \approx 0 \rightarrow \text{Normal tails}$; $> 2 \rightarrow \text{heavy tails (outlier risk)}$; $< -1 \rightarrow \text{flat tails}$

Step 4: Apply the right transform (log, Box-Cox, sqrt) to reduce skewness

Step 5: Re-check skewness and kurtosis after transform

Step 6: Run Shapiro-Wilk test to confirm normality after transform

6 All Chart Types — Explained with Python

Why Different Charts?

Different data types and questions need different visualisations.

Using the wrong chart is like using a spoon to cut bread — technically possible but wrong!

Rule of thumb:

Distribution of data? → Histogram or Box Plot

Compare categories? → Bar Chart

Relationship between variables? → Scatter Plot

Change over time? → Line Chart

Histogram — The Distribution Detective

Groups data into bins and shows how many values fall in each bin. Best for: understanding the SHAPE and DISTRIBUTION of a single numerical variable.

Histogram vs Bar Chart — Key Difference

Histogram: X-axis is NUMERICAL (continuous). Bars touch each other (no gap).

Used for one numerical variable's distribution.

Bar Chart: X-axis is CATEGORICAL. Bars have gaps between them.

Used to compare values across different categories.

Easy trick: if bars touch each other → Histogram. Gaps between bars → Bar Chart.

Python Code

```
import matplotlib.pyplot as plt, seaborn as sns, numpy as np

data = np.random.normal(70, 15, 500) # exam scores

plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.hist(data, bins=20, color='steelblue', edgecolor='white')
plt.title('Histogram - Exam Scores')

plt.subplot(1, 2, 2)
sns.histplot(data, kde=True, color='steelblue', bins=20)
plt.title('Histogram + KDE (seaborn)')
plt.tight_layout(); plt.show()
```

□ Box Plot — The Outlier Detector

Shows the 5-number summary: Min, Q1, Median, Q3, Max — plus outliers as dots. Best for: comparing distributions across groups and spotting outliers instantly.

□ Python Code

```
import matplotlib.pyplot as plt, seaborn as sns, numpy as np, pandas as pd

data = { "Math": np.random.normal(70,10,100), "Science":
np.random.normal(65,15,100),
        "English": np.random.normal(75,8,100) }
df = pd.DataFrame(data)
df_melt = df.melt(var_name='Subject', value_name='Score')

sns.boxplot(x='Subject', y='Score', data=df_melt)
plt.title('Box Plot - Subject Scores')
plt.show()

# Reading a box plot:
# Bottom whisker = Min (or 1.5×IQR below Q1)
# Box bottom    = Q1  (25th percentile)
# Middle line   = Median
# Box top       = Q3  (75th percentile)
# Top whisker   = Max (or 1.5×IQR above Q3)
# Dots          = Outliers
```

□ Bar Chart

□ Python Code

```
import matplotlib.pyplot as plt, seaborn as sns, pandas as pd

subjects = ['Math', 'Science', 'English', 'History']
avg_score = [72, 68, 75, 60]
df = pd.DataFrame({'Subject': subjects, 'Average': avg_score})

sns.barplot(x='Subject', y='Average', data=df, palette='Blues_d')
plt.title('Bar Plot - Average Scores'); plt.show()
```

□ Line Chart — For Trends Over Time

□ Python Code

```
import matplotlib.pyplot as plt

months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun']
sales = [120, 135, 148, 162, 155, 178]
```

```
plt.plot(months, sales, marker='o', color='steelblue', linewidth=2)
plt.fill_between(months, sales, alpha=0.2, color='steelblue')
plt.title('Monthly Sales - Line Chart'); plt.grid(True, alpha=0.3); plt.show()
```

□ Scatter Plot — Relationship Between Two Variables

□ Python Code

```
import matplotlib.pyplot as plt, seaborn as sns, numpy as np

height = np.random.normal(170, 10, 200)
weight = height * 0.4 + np.random.normal(0, 5, 200)

sns.regplot(x=height, y=weight, scatter_kws={'alpha':0.4})
plt.xlabel('Height (cm)'); plt.ylabel('Weight (kg)')
plt.title('Scatter + Regression Line'); plt.show()
```

□ Other Important Charts

Chart	Best For	Python
Pie Chart	Show proportions/percentages of a whole	plt.pie(values, labels=labels)
Violin Plot	Like box plot but shows full distribution shape too	sns.violinplot(x, y, data=df)
Heatmap	Show correlation matrix or 2D data as colour grid	sns.heatmap(df.corr(), annot=True)
Pair Plot	All scatter plots between all variable pairs at once	sns.pairplot(df)
KDE Plot	Smooth version of histogram (probability density)	sns.kdeplot(data)
Q-Q Plot	Check if data is Normal	stats.probplot(data, plot=plt)
Count Plot	Count of each category (like bar for categoricals)	sns.countplot(x='col', data=df)

□ All Charts — Quick Decision Guide

Which Chart Should I Use?

Distribution of ONE numerical variable?

- Histogram (shape, skew, outliers)
- KDE Plot (smooth histogram)

- Box Plot (IQR, outliers, 5-number summary)
- Violin (box plot + density shape)

Compare VALUES across CATEGORIES?

- Bar Chart (single comparison)
- Grouped Bar (multiple groups)

Show PROPORTIONS of a whole?

- Pie Chart (use sparingly — hard to read accurately)

Relationship between TWO numerical variables?

- Scatter Plot
- Scatter + Regression Line

CHANGE OVER TIME?

- Line Chart

Correlation between MANY variables?

- Heatmap (correlation matrix)
- Pair Plot (all scatter plots at once)

Check if data is NORMAL?

- Q-Q Plot + Histogram with KDE
- Shapiro-Wilk test (code above)

⚡ Final Cheat Sheet — Everything at a Glance

Data Types

Type	Sub-type	Example	Python
Categorical	Nominal	Blood type, country	pd.Categorical / object
Categorical	Ordinal	Star ratings, grade	pd.Categorical(ordered=True)
Numerical	Discrete	Number of goals	int64
Numerical	Continuous	Height, weight	float64
Numerical	Interval	Temperature °C	float64
Numerical	Ratio	Income, height cm	float64

Distributions

Distribution	Type	Parameter	Real-World Use
Normal / Gaussian	Continuous	μ, σ	Heights, errors, ML weight init
Uniform	Both	a, b	Fair dice, random number generation
Bernoulli	Discrete	p	Single coin flip, spam/not spam
Binomial	Discrete	n, p	n coin flips — how many heads?
Poisson	Discrete	λ (rate)	Events per time unit (emails/hr)
Exponential	Continuous	λ	Time BETWEEN events (wait time)
T-distribution	Continuous	df (degrees)	Small sample testing (n<30)

Skewness & Kurtosis Quick Reference

Metric	Value	Meaning	Action
Skewness	≈ 0	Symmetric — Normal-like	No action needed
Skewness	$> +0.5$	Right-skewed (positive)	Apply log or Box-Cox transform
Skewness	< -0.5	Left-skewed (negative)	Reflect + log, or Yeo-Johnson

Metric	Value	Meaning	Action
Kurtosis (excess)	≈ 0	Normal tails	Safe to use Normal assumptions
Kurtosis (excess)	> 1	Heavy tails (leptokurtic)	Expect outliers, use robust methods
Kurtosis (excess)	< -1	Light tails (platykurtic)	Data is more uniform, fewer extremes

Charts

Chart	Best For	Code
Histogram	Distribution of one numerical var	<code>plt.hist(data) / sns.histplot(data, kde=True)</code>
Box Plot	Outliers, IQR, compare groups	<code>sns.boxplot(x, y, data=df)</code>
Bar Chart	Compare categories	<code>plt.bar(cats, vals) / sns.barplot()</code>
Scatter Plot	Relationship between 2 variables	<code>plt.scatter(x, y) / sns.regplot()</code>
Line Chart	Change over time	<code>plt.plot(x, y, marker='o')</code>
Violin Plot	Distribution shape per group	<code>sns.violinplot(x, y, data=df)</code>
Heatmap	Correlations, 2D data	<code>sns.heatmap(df.corr(), annot=True)</code>
Q-Q Plot	Check if Normal	<code>stats.probplot(data, plot=plt)</code>

— End of Notes —